

Pearl Floating-Point Scheme Specification: FP16 Accumulation-Hardness Variant

Jon Pry
jonpry@gmail.com

October 2026

Abstract

This document specifies an alternative instantiation of the Pearl proof-of-useful-work protocol in which the unit of mining work is FP16 matrix multiplication on GPUs, with NVIDIA A100 (GA100) tensor cores as the first whitelisted device. It extends the FP8 certificate-v4 specification rather than replacing it: the commitment structure, seed chain, state window, ticket, target, mixture-of-experts extension, and zero-knowledge split are unchanged, and the FP8 scheme remains available as a separate `Quant` value. The new content is a *hardness source that does not rely on coarse quantization*. Where the FP8 scheme makes the lottery product hard to shortcut by rounding onto a coarse grid, the FP16 scheme derives hardness from the *nonlinearity of the device’s own accumulation*: the A100 tensor core truncates each product onto a per-group alignment grid before summing, an operation that does not commute with the reduction and therefore cannot be expressed as a matrix multiplication. We give the bit-exact A100 accumulation model (validated against silicon), a policy check—*unpredictable accumulation steps*—that the verifier evaluates during its tile replay, and an analysis of the attack surface. The practical payoff is that recovered products carry FP16-level accuracy rather than FP8-residual accuracy, at the cost of a new, device-specific hardness assumption. We find FP16 robust and BF16 too close to linear to admit the same argument.

Contents

1	Introduction	1
2	The abstract protocol	2
3	Preliminaries	3
4	The A100 accumulation model	3
5	Hardness	4
6	Jackpot policy	5
7	Attack vectors	6
8	Honest miner overview	7
9	FP16 vs. FP8 vs. BF16	7

A A100 arithmetic specification	8
B FP16 commit type	8
C Noise and noisy quantization	8
D Certified-work ratio	8
E Empirical validation	9

1 Introduction

Proof-of-useful-work (PoUW) replaces the artificial puzzle of classical proof of work with a computation someone already wants performed. The Pearl protocol instantiates the PoUW of Komargodski and Weinstein [1] for matrix multiplication: a miner multiplies matrices of its own choosing and every tile of the product doubles as a lottery ticket. The obstacle is permissionlessness—the miner chooses its inputs, so the protocol must guarantee that no choice of inputs, however degenerate, makes valid tickets cheaper than the prescribed amount of work.

The original integer protocol [1] achieves this by hashing the *transcript* of the computation: the product is formed in $r \times r$ blocks and a random oracle is applied to all $(n/r)^3$ intermediate blocks, so that even the all-zeroes product is as hard as a generic one. The Pearl FP8 specification [2] instead hashes only output tiles and recovers hardness from quantization: because entrywise quantization is nonlinear, the low-rank noise injected before quantization destroys the low-rank shortcut, and tiles can be hashed directly. This is cheaper than transcript hashing, but its security depends on the rounding being coarse: the quantization error must be large enough that its contribution to the output survives the accumulator. At FP8 this holds comfortably; as the target format becomes more precise, the quantization error shrinks and the margin vanishes.

This document describes a third point in the design space, aimed at FP16. We keep output-tile hashing—so the miner runs its ordinary GEMM with no in-kernel hashing—but we obtain hardness from a different nonlinearity. Modern tensor cores do not accumulate in exact arithmetic. On the A100, each product is truncated toward zero onto an alignment grid shared by a group of eight products, the group is summed, and the running accumulator is rounded after every group. The truncation is applied *per product, before the reduction*. This is the same structural device that makes transcript hashing hard—a per-summand operation that a single contraction cannot reproduce—except that it is performed by the hardware for free, on every multiply-accumulate the miner already issues. An honest miner’s output therefore encodes, at no extra cost, a quantity that an adversary with the low-rank shortcut cannot cheaply reconstruct.

Organization. Section 2 recalls the abstract protocol and states the new hardness premise. Section 3 fixes notation and the work model. Section 4 gives the bit-exact A100 accumulation model—the technical core of this variant. Section 5 states the hardness argument and assumption. Section 6 specifies the jackpot policy, including the unpredictable-accumulation-steps check. Section 7 surveys attack vectors. Section 8 describes the honest miner. Section 9 compares FP16, FP8 and BF16. The appendices give the A100 arithmetic specification, the FP16 commit type, the noise and quantization procedures, the certified-work ratio, and a summary of the empirical validation.

2 The abstract protocol

Let $A \in \mathbb{R}^{m \times k}$ and $B \in \mathbb{R}^{n \times k}$, with useful product $C := AB^\top$. As in the FP8 scheme, the protocol derives seed-dependent rank- r noise matrices $N_A \in \mathbb{R}^{m \times k}$ and $N_B \in \mathbb{R}^{n \times k}$ and computes

$$\tilde{C} := Q(A + N_A) Q(B + N_B)^\top, \quad (1)$$

where Q is the protocol’s quantization into the committed storage format and the multiplication is carried out by the committed device’s native kernel. Each noise matrix is unpredictable before the inputs are committed. The product $\tilde{C}$ drives the lottery; the honest miner recovers the useful product by the cheap low-rank correction $\hat{C} := \tilde{C} - AN_B^\top - N_AB^\top - N_A N_B^\top$, exactly as in [1, 2].

The difference from the FP8 scheme is in *what makes $\tilde{C}$ hard to shortcut*. Setting $A = B = 0$ isolates the lottery product as $Q(N_A) Q(N_B)^\top$. The FP8 premise is that entrywise quantization raises the rank of $Q(N_A)$ enough that this product is as hard as a generic quantized product. The FP16 premise is weaker on quantization and stronger on the device: at FP16 the quantization barely raises the rank, but the device does not compute the exact product $Q(N_A)Q(N_B)^\top$ —it computes a *per-product-truncated* version of it, tile by tile, and it is that version the ticket hashes.

The central object. For operands stored in a format with p -bit significands, write the device dot product of row i of $\tilde{A} := Q(A + N_A)$ with row j of $\tilde{B} := Q(B + N_B)$ as

$$D_{ij} = \text{Acc}(\tilde{A}_{i,0}\tilde{B}_{j,0}, \dots, \tilde{A}_{i,k-1}\tilde{B}_{j,k-1}), \quad (2)$$

where Acc is the device accumulation of Section 4, *not* the exact sum. The exact sum $S_{ij} = \sum_u \tilde{A}_{iu}\tilde{B}_{ju}$ is, for low-rank-plus-noise operands, computable by an adversary in $O(r)$ work per cell through the low-rank structure. The quantity D_{ij} differs from $\text{RZ}(S_{ij})$ by the accumulated per-product truncation remainders, and reconstructing those remainders is what the protocol charges for. Section 5 makes this precise.

3 Preliminaries

Notation. Matrices are over the reals, stored in a committed floating-point format. $C := AB^\top \in \mathbb{R}^{m \times n}$; C_{ij} is the inner product of row i of A with row j of B . For selected row-index sets I_A, I_B (chosen by periodic partitions $\mathcal{P}_A, \mathcal{P}_B$, as in the FP8 scheme), $\tilde{C}_{I_A, I_B} := \tilde{A}_{I_A}(\tilde{B}_{I_B})^\top$ is the recomputed tile. The seeds `noise_seedA`, `noise_seedB` determine the noise. The ticket digest is a keyed function `Ticket` from tiles to $\{0, 1\}^\lambda$, $\lambda = 256$:

$$\text{Ticket}(\tilde{C}_{I_A, I_B}) := \text{H}^{\text{“jackpot”}}(\text{Extract}(\tilde{C}_{I_A, I_B}, \mathcal{P}_A, \mathcal{P}_B); \text{noise_seed}_A).$$

All of `Ticket`, `Extract`, the seed chain, the state window of depth D , the domain-separated BLAKE3 hashing, and the commitment format are shared with the FP8 specification [2] and are not restated here except where FP16 changes them (Appendix B).

Work model. Work is measured in multiply-accumulate operations (MACs). The direct cost of the selected tile is $T_{\text{tile}} := |I_A| \cdot |I_B| \cdot k$. As in [2] the MAC count fixes scale only; it excludes quantization, noise generation, hashing, and memory movement. The novelty of this variant is that the quantity securing the tile is produced *inside* these MACs by the device, so no work beyond the GEMM is charged to the miner.

Device and Quant. Device and Quant are consensus-allowed enumerations. This document adds Device = A100 with the accumulation model of Section 4, and a Quant value whose storage type is FP16 and whose commit type is the FP16 block format of Appendix B. The FP8 values remain available unchanged.

4 The A100 accumulation model

We specify the arithmetic of the A100 `HMMMA.16816.F32` instruction (and the equivalent `m16n8k8/m16n8k16` and `wmma` forms, which chain identically) for FP16 and BF16 inputs with FP32 accumulation. The model is validated bit-exact against silicon in Appendix E. The verifier’s implementation is authoritative; the description below is phrased in exact arithmetic.

Let the k -axis be partitioned into *groups* of $G = 8$ consecutive products, processed in order (two groups per k16 instruction). The internal accumulator precision is $W = 24$ significant bits (full FP32 significand). For a stored value x , let $\varepsilon(x)$ be its unbiased stored exponent, with subnormal inputs taking the format minimum (-14 for FP16, -126 for BF16). For a nonzero FP32 accumulator c , let $\varepsilon(c) := \max(\lfloor \log_2 |c| \rfloor, -126)$. Let $\text{Trunc}(x, g)$ denote truncation of x toward zero onto the grid 2^g (sign-magnitude).

One group step. With incoming accumulator c and group products $p_u = \tilde{A}_{iu}\tilde{B}_{ju}$ (each exact; its exponent is $\varepsilon(\tilde{A}_{iu}) + \varepsilon(\tilde{B}_{ju})$):

1. If the group has no nonzero product and c is unchanged, the step is a no-op.
2. Compute the alignment exponent $\eta := \max(\{\varepsilon(\tilde{A}_{iu}) + \varepsilon(\tilde{B}_{ju}) : p_u \neq 0\} \cup \{\varepsilon(c)\})$, where $c = 0$ does not participate and a subnormal c contributes -126 .
3. Truncate every product and the accumulator onto the grid $2^{\eta-W}$: $\bar{p}_u := \text{Trunc}(p_u, \eta - W)$, $\bar{c} := \text{Trunc}(c, \eta - W)$.
4. Sum the integers exactly: $S := \bar{c} + \sum_u \bar{p}_u$.
5. Round S toward zero to FP32: $c' := \text{RZ}_{\text{FP32}}(S)$, i.e. keep the top 24 significant bits, place subnormal results on the 2^{-149} grid, and map $|S| \geq 2^{128}$ to $\pm\infty$.

The final accumulator after all groups is D_{ij} . Boundary behavior: a zero result is always $+0$; ∞ and NaN follow IEEE-754 on the inputs (NaN sign and payload are unspecified); an overflowed accumulator is *sticky*—a later opposite-sign overflow does not produce NaN, and the first overflow’s sign is retained. The verifier rejects any tile whose operands or intermediates are non-finite (Appendix A).

The key feature. Step 3 truncates each product *individually* before step 4 sums them. Write p' for the product significand width (the product of two p -bit significands carries $p' = 2p$ significant bits: $p' = 22$ for FP16, $p' = 16$ for BF16). Once the running accumulator exceeds a group’s products by more than about $W - p'$ bits—roughly a factor $2^{W-p'}$ —the low bits of each product fall below the grid $2^{\eta-W}$ and are discarded. For FP16, $p' = 22$ and $W = 24$, so a product loses bits as soon as the accumulator is more than $\sim 4\times (2^{W-p'} = 2^2)$ its magnitude; for a realistic dot product this happens for the majority of products. For BF16, $p' = 16$, so products survive the window unless they are more than $W - p' = 8$ binades below η ; far fewer are truncated (Section 9).

Breakpoints. Call a group step a *breakpoint* if the accumulator truncation in step 3 or the round in step 5 discards a nonzero bit, i.e. if $\bar{c} \neq c$ or $\text{RZ}_{\text{FP32}}(S) \neq S$. Per-product truncations ($\bar{p}_u \neq p_u$) are *not* breakpoints; they are counted separately as N_{pt} (Section 6), which is why N_{pt} is well defined as the truncated products in non-breakpoint steps. The density of breakpoints over a tile,

f_{bp} , measures how much of the accumulation is irrecoverable from the exact sum alone; it is the quantity the policy of Section 6 bounds from below.

5 Hardness

Exact decomposition. For any inputs, the device output admits the exact decomposition

$$D_{ij} = \mathcal{R}\left(S_{ij} - \sum_g \sum_{u \in g} \rho_u - \sum_g \gamma_g\right), \quad (3)$$

where $S_{ij} = \sum_u p_u$ is the exact sum, $\rho_u = p_u - \text{Trunc}(p_u, \eta_g - W)$ is the per-product truncation remainder in group g , γ_g is the accumulator re-truncation and per-group rounding remainder, and $\mathcal{R}$ denotes the per-group RZ chain. (This reproduces the model of Section 4 bit-for-bit; see Appendix E.) For low-rank-plus-noise operands the adversary obtains S_{ij} —and prefix sums of it, hence every η_g —in $O(r)$ work per cell. The hardness therefore rests entirely on the correction terms $\sum_u \rho_u$ and $\sum_g \gamma_g$.

Non-contractibility. The decisive property is that the correction is a *per-product nonlinearity applied before the reduction*. The map $p_u \mapsto \text{Trunc}(p_u, \eta_g - W)$ does not commute with summation:

$$\sum_u \text{Trunc}(p_u, \eta_g - W) \neq \text{Trunc}\left(\sum_u p_u, \eta_g - W\right) \quad \text{in general.}$$

Consequently $\sum_u \rho_u$ is not a function of the contracted sum S_{ij} , and it is not itself a bilinear form in the operand rows. No matrix multiplication—FP16, integer, or binary—evaluates it: a contraction can produce $\sum_u p_u$ or any fixed linear image of the products, but not the sum of a nonlinear, exponent-dependent function applied to each product. The corrections must be computed element-wise, on the mk (resp. nk) products, outside the tensor-core datapath. This is the same obstruction that makes the integer transcript scheme hard [1]; here the device applies it automatically to every MAC.

Assumption. We formalize the resulting premise, in the style of [1, Assumption 6.4] and the FP8 transcript-unpredictability assumption.

Assumption 1 (Accumulation-unpredictability). *Fix Device = A100, Quant with FP16 storage, noise rank $r = 32$, and admissible tile dimensions. For operands drawn as in Appendix C (low-rank noise added before quantization, any miner-chosen A, B admitted by the policy of Section 6), no algorithm reconstructs the device tile $\{D_{ij}\}$ bit-exactly in expected time substantially below the honest cost $|I_A| |I_B| k$ MACs. In particular the corrections $\sum_u \rho_u, \sum_g \gamma_g$ of (3) cannot be batched onto any tensor-core path.*

Evidence. Appendix E summarizes the experiments supporting Assumption 1: (i) the exact model is validated on A100 silicon over $> 10^7$ dot products; (ii) for every input strategy tried, including sixteen adversarial constructions, the FP16 breakpoint density stays in $[0.36, 0.51]$ and a deliberately attacker-favorable certified-work ratio (Appendix D) stays ≥ 1.38 ; (iii) the cheapest shortcut predictors (exact sum rounded to FP32; group sums without per-product truncation) reproduce at most a few percent of cells; (iv) the truncation-correction attack, costed concretely, is $6\text{--}13\times$ honest even under compute-bound-optimistic assumptions, because the corrections run on CUDA cores rather than tensor cores. Assumption 1 is a device-specific hardness conjecture and should be treated as such.

6 Jackpot policy

Beyond the idealized setting, the miner may choose A, B to make the accumulation cheaply predictable—chiefly by arranging that few products are truncated. The verifier defends with a pointwise policy evaluated *during the bit-exact tile replay* of Section 4, when all intermediate η_g , $\bar{p}_u$, $\bar{c}$, and S are available at no extra cost. The policy approves exactly when every check below passes; it runs after the Infinity/NaN rejection.

Recorded during replay. For each cell (i, j) and group g the verifier records:

- $\text{bp}_{ijg} \in \{0, 1\}$: whether the step is a breakpoint (Section 4);
- $\text{ptc}_{ijg} \in \{0, \dots, G\}$: the number of products in the group with a nonzero truncation remainder ρ_u .

Shared checks. The FP8 entry-liveness and noise-floor checks carry over to bound degenerate operands:

$$\sigma_i^A \geq \sigma_{\min} \ (\forall i \in I_A), \quad \sigma_j^B \geq \sigma_{\min} \ (\forall j \in I_B), \quad |\mathcal{D}_A| \leq \varepsilon_{\text{idle}} |I_A| k, \quad |\mathcal{D}_B| \leq \varepsilon_{\text{idle}} |I_B| k,$$

with $\sigma_{\min} = 1$, $\varepsilon_{\text{idle}} = 1/64$, $\tau_{\text{idle}} = 8$ and the idle sets $\mathcal{D}_X$ as in [2]. These keep every opened row carrying at least one quantized unit of noise and bound the fraction of noise-dominated entries.

Unpredictable accumulation steps (new). Define the tile breakpoint density and the certified-work ratio

$$f_{\text{bp}} := \frac{1}{|I_A||I_B|G^{-1}k} \sum_{i,j,g} \text{bp}_{ijg}, \quad \rho := \frac{1}{|I_A||I_B|k} \sum_{i,j} \left[G \cdot N_{\text{bp}}^{ij} + r \cdot N_{\text{run}}^{ij} + N_{\text{pt}}^{ij} \right],$$

where N_{bp}^{ij} is the number of breakpoint steps in cell (i, j) , N_{run}^{ij} is the number of maximal runs of non-breakpoint steps, and $N_{\text{pt}}^{ij} = \sum_g \text{ptc}_{ijg}$ counts truncated products outside breakpoints. The ratio ρ is the attacker-favorable lower bound on reconstruction cost derived in Appendix D. The check requires

$$f_{\text{bp}} \geq 0.30, \quad \rho \geq 1.2. \tag{4}$$

Calibration. On A100 FP16, honest and realistic workloads sit at $f_{\text{bp}} \in [0.44, 0.48]$ and $\rho \in [1.64, 1.83]$, with f_{bp} concentrating to within 0.003 across noise draws, so (4) is a safety net rather than an active constraint: every adversarial construction tested also passes it, the worst observed being $f_{\text{bp}} = 0.357$, $\rho = 1.38$ (Appendix E). Because the check is a density gate over the whole tile, honest tiles do not fail on fluctuations, and because it concentrates, grinding noise draws yields no advantage on honest-like data. BF16 does not admit a separating threshold of this form at $k \leq 1024$ (Section 9); the FP16 thresholds (4) apply only to the FP16 Quant value.

7 Attack vectors

Low-rank inputs. A miner may choose low-rank A, B so that S_{ij} is cheap. This is allowed; hardness does not depend on the exact sum being hard. By (3) the device output still requires the per-product corrections, which are non-contractible.

Truncation / bit-slice correction. The corrections ρ_u are shallow (median ~ 4 discarded bits per FP16 product), which suggests batching them on binary or int8 tensor cores via bit-planes. This fails for the structural reason of Section 5: bit-planes accelerate *contractions*, and the correction is not a contraction. Even granting the adversary free exact prefix sums and a free oracle for which steps are breakpoints, the corrections must be evaluated elementwise on CUDA cores, costing $6\text{--}13\times$ honest work under compute-bound-optimistic accounting and far more when memory-bound (Appendix E).

Exact-sum shortcut. Computing $\text{RZ}_{\text{FP32}}(S_{ij})$ and returning it reproduces the device output in a negligible fraction of cells ($\leq 1.6\%$ in all FP16 tests), because the per-product truncation changes almost every cell.

Minimizing truncation. The miner may seek operands with flat exponents or few significant bits to reduce f_{bp} . For FP16 this is self-defeating: the mandated noise (rank $r = 32$, relative scale $\delta = 1/2$) spreads product exponents enough that truncation remains dense, and the policy (4) rejects any tile that nonetheless falls into a low-truncation regime. For BF16 the same strategy succeeds, which is why BF16 is excluded (Section 9).

Device emulation and ASICs. Reproducing A100 accumulation on another GPU requires emulating the $G = 8$ grouping and per-product truncation, which that device’s tensor cores cannot do cheaply; it must fall back to the same elementwise path as any attacker. A custom ASIC could implement the per-product correction more cheaply than a full FP16 multiplier, so the argument is against commodity GPUs, not arbitrary silicon; this is the same economic question raised for the FP8 scheme and in the Pearl open-problems note.

Seed and commitment grinding. The noise is seed-derived after commitment; grinding seeds only yields additional lottery tickets, bounded by the usual Fiat–Shamir argument.

8 Honest miner overview

The intended miner mines with FP16 multiplications it wants computed anyway.

1. **Prequantize.** Encode A, B in the FP16 commit format (Appendix B).
2. **Commit.** Hash B under a recent header and derive `noise_seedB`; commit A under the proposed header and derive `noise_seedA`.
3. **Noise and quantize.** Derive the rank- r noise and compute $\tilde{A} = \text{NoisyQuantize}(A, \dots)$, $\tilde{B} = \text{NoisyQuantize}(B, \dots)$ in $O((m+n)kr)$ MACs (Appendix C).
4. **Multiply.** Run the A100 FP16 kernel once on the noised operands— mnk MACs, the dominant cost, identical to the kernel the job would run without the protocol.
5. **Scan for winners.** Cut the output into tiles, compute `Extract` and the jackpot hash per tile, and compare against the scaled target.
6. **Open on a win.** For a winning tile admitted by the policy, assemble the proof.

Recovering the useful product costs the low-rank correction plus the $O((mn + mk + nk)r)$ overhead of hashing, noising and scanning, for a $1 + o(1)$ total overhead when $m, n, k \gg r$. The recovered product carries FP16-level accuracy: the device truncation that secures the tile is of the same order as—and is reconstructed together with—the quantization error, so no accuracy beyond ordinary FP16 rounding is sacrificed. Critically, the miner’s kernel must use the committed instruction

order with no split- k or atomic accumulation, since any reordering changes the bit-exact output (Appendix A).

9 FP16 vs. FP8 vs. BF16

Why FP16 works. FP16 products carry 22 significant bits against the 24-bit accumulator window, so truncation is dense and, crucially, insensitive to the miner’s choices: across all tested strategies $f_{bp} \in [0.36, 0.51]$. The non-contractibility argument then applies with a wide margin.

Why BF16 does not. BF16 products carry only 16 bits and fit inside the 24-bit window unless more than 8 binades below η . Flat rank-one sign-pattern operands leave $>99\%$ of products exact and only 6–8% of steps as breakpoints; a groupwise model without per-product truncation already reproduces more than half the outputs, and the certified-work ratio falls to $\rho \approx 0.30$. These operands pass the liveness and noise-floor checks. Honest BF16 is itself marginal ($\rho \in [0.84, 1.10]$ at $k = 1024$), so no threshold separates honest from adversarial BF16 with margin. BF16 mining, if desired, requires either FP16 storage, $k \geq 4096$, or a larger noise rank; this document does not specify it.

Relation to FP8. The FP8 scheme remains the cheapest option and its hardness margin is ample; it is preferred for workloads tolerant of FP8-residual accuracy. FP16 trades a modest constant factor (denser truncation to reconstruct, a slightly larger proof) for FP16-level output accuracy and a hardness source independent of the rounding grid. A deployment may offer both Quant values and weight their targets so that each is mined at its true device throughput; the economics mirror the FP8 discussion in [2].

A A100 arithmetic specification

Element-wise FP16/BF16/FP32 operations follow IEEE-754-2019 with the roundings used by the device: casts to FP16 round to nearest even and saturate per the format; the matrix-multiply accumulation is specified in Section 4. The verifier rejects NaN and infinity in operands and in every intermediate, applying the test after rounding. The A100 accumulator keeps $W = 24$ significant bits (no sub-FP32 truncation at the accumulator level, unlike the Hopper/Blackwell FP8 paths of [2]); the nonlinearity is entirely in the per-product, per-group truncation of Section 4. The `m16n8k8` instruction is identical to `m16n8k16` with $G = 8$; the FP16-accumulate variant (`HMMMA.16816.F16`) uses the same grouping and truncation followed by round-to-nearest-even to FP16 per group and is not used by this scheme.

B FP16 commit type

An operand with n rows is committed as a single matrix $V \in \text{FP16}^{n \times k}$, each row a Merkle leaf-set under the key and hash identifier of the main protocol, with aggregate root H_X . Unlike the FP8 FP10 commit (int8 values plus a block scale), the FP16 format stores values directly, since FP16 already carries the target precision; no block scale is required. Row norms $\ell_2(X_t), \ell_\infty(X_t)$, used by the noisy-quantization scales, are computed from V in FP32 and floored as in [2].

C Noise and noisy quantization

The noise sampler, seed chain, and rank- r construction $N = EF^\top$ are those of the FP8 specification, with $r = 32$ and per-row relative scale $\delta = 1/2$. Noisy quantization computes, per row, $\tilde{A}_t = \text{RNE}_{\text{FP16}}(\alpha_t X_t + \beta_t N_t)$ with the scales α_t, β_t chosen so the noise carries relative Euclidean weight δ ; the clamp and norm-floor rules follow Appendix C.4 of [2] with target magnitude Q set to the largest finite FP16 value 65504. Because FP16 is the target precision, α_t performs only range normalization and the recovered product retains FP16 accuracy after the low-rank correction.

D Certified-work ratio

The ratio ρ of Section 6 is an attacker-favorable lower bound on the cost of reconstructing a tile under the following generous premises: (i) exact prefix sums over any k -range cost r MAC-equivalents per cell via the low-rank structure; (ii) a run of consecutive non-breakpoint steps can be merged at cost r plus one per truncated product; (iii) each breakpoint step costs $\min(G, r) = G$ to resolve its discarded low bits from the previous exact state; (iv) the breakpoint locations are supplied for free by an oracle. Summing gives the per-cell cost $G N_{\text{bp}} + r N_{\text{run}} + N_{\text{pt}}$ used in the definition, normalized by the honest k . Premises (i)–(iv) overstate the attacker’s power—in particular, quantization makes the operands only approximately low rank—so a measured $\rho \geq 1.2$ is a conservative certificate that no such shortcut beats honest work.

E Empirical validation

Scope of the bit-exactness claim. The accumulation model is validated bit-exact *only* on GA100 (sm_80) silicon. Other Ampere/Ada parts (sm_86, sm_87, sm_89) are distinct tensor-core designs whose intra-instruction accumulation has not been characterized against this model—they may or may not agree—so the whitelisted device is GA100 (sm_80) exactly, and the miner admits that capability alone (fail-closed). Whitelisting any further SKU requires repeating the validation below on that part.

All experiments were run on A100-class (GA100, sm_80) silicon. The accumulation model of Section 4 matched the hardware bit-for-bit on $> 1.1 \times 10^6$ random dot products per device (both devices agreeing), on 69 directed edge-case scenarios totaling 6.9×10^6 dot products (subnormal inputs and accumulators, exact cancellation, mixed signs, grid-boundary products, partial and empty groups, $k = 2048$ chains, overflow, infinity, NaN, and $0 \cdot \infty$), and on the adversarial and realistic noised operands below. One residual is known: BF16 chains whose partial sums remain entirely in the FP32-subnormal range can be off by one subnormal ulp in $< 1\%$ of outputs; this cannot occur for FP16 or for $O(1)$ -scaled operands and is excluded by policy.

Adversarial study (tiles 16×16 , $k = 1024$, $r = 32$, $\delta = 1/2$, operands row-scaled to unit max magnitude): sixteen input strategies—realistic Gaussian with outlier channels, heavy-tailed, $A = B = 0$, rank-two, several flat and few-bit constructions designed to minimize truncation, sparse, spiked, and geometric—all kept FP16 at $f_{\text{bp}} \in [0.36, 0.51]$ and $\rho \geq 1.38$, with the cheapest shortcut predictors reproducing $\leq 1.6\%$ of cells (one geometric case 13%, still $\rho = 1.68$). The same strategies drive BF16 to f_{bp} as low as 0.06 and $\rho \approx 0.30$. The truncation-correction attack, costed end-to-end, is 6–13 $\times$ honest (compute-bound-optimistic) and 220–470 $\times$ (measured, memory-bound), because the per-product corrections cannot use tensor cores.

Reproducible harness. A self-contained harness is committed alongside this specification at `docs/fp16_scheme/validation/` (it links only the CUDA runtime, so it runs on an `sm_80` box without the miner’s Python stack). On A100-class silicon (validated on a GA100 `sm_80` device) it: (i) captures the output of the real `mma.sync.m16n8k16.f32.f16` and checks the model of Section 4 against the device *bit-for-bit* over 910 dot products—the 238 committed reference vectors plus 272 generated edge cases (k up to 1024, subnormal operands, subnormal accumulators, subnormal outputs, near-total cancellation, grid-boundary, and max-magnitude)—with zero mismatches, so the model is validated against the device rather than against another copy of itself; (ii) resolves the rounding mode on silicon, *confirming round-toward-zero* at both the per-product alignment and the per-group FP32 encode: the device matches the RZmodel on all 400 vectors constructed to separate RZ from round-to-nearest-even, while each RNE variant disagrees with the device on a large fraction of them; (iii) recomputes the policy metrics f_{bp}, ρ and the shortcut-predictor rates over the adversarial strategies, through both an independent noise implementation and the exact consensus noise pipeline, reproducing the regime above ($f_{bp} \in [0.45, 0.52]$, $\rho \in [1.67, 1.94]$, cheapest shortcut $\leq 1.6\%$); and (iv) micro-benchmarks the truncation-correction attack against the honest tensor-core FP16 GEMM (the miner’s own kernel), measuring $\approx 38\text{--}43\times$ honest on this silicon even when the attacker is granted the exact sums and the breakpoint oracle for free—consistent with the accounting of Section 7. The model-vs-silicon corpus is enforced in CI by the Rust test `api::fp16::accumulate::matches_hardware_capture_k1024`; the overflow guard was never triggered, confirming that a single FP16 dot product cannot overflow.

References

- [1] I. Komargodski and O. Weinstein. *Proofs of Useful Work from Arbitrary Matrix Multiplication*. arXiv:2504.09971; IACR ePrint 2025/685, 2025.
- [2] Pearl Research Team. *Pearl Floating Point Scheme Specification* (FP8 certificate-v4), September 2026.
- [3] E. Badash, D. Boneh, I. Komargodski, and M. Srivastava. *Hawkeye: Reproducing GPU-level Non-determinism*. arXiv:2603.20421, 2026.
- [4] J. O’Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O’Hearn. *BLAKE3: one function, fast everywhere*, 2020.